Experiment 10

Student Name: Simran Chouhan
UID: 24BAI70124
Branch: CSE - AIML
Section/Group: 24AIT_KRG G1
Semester: 4
Date of Performance: 1/4/26
Subject Name: DBMS
Subject Code: 24CSH-298

Aim

To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

Objectives

To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

Practical/Experiment Steps

Identify the table requiring automated primary key enforcement.

Design a trigger that activates before insert operations.

Ensure that every new record receives a unique primary key automatically.

Validate the trigger by inserting multiple records and verifying unique keys.

Procedure

• Accessed the PostgreSQL console and created the employees table with fields for emp_id, name, and salary, ensuring emp_id is defined as the primary key.

• Created a sequence (emp_seq) to generate unique incremental values for the primary key.

• Defined a trigger function (emp_trigger_function) using the NEW keyword to access and modify incoming row data before insertion into the table.

• Implemented logic inside the function to automatically assign emp_id using nextval('emp_seq') when no value is provided during insertion.

• Created the emp_trigger trigger and configured it as BEFORE INSERT and FOR EACH ROW to ensure every new record is processed individually.

• Executed INSERT statements without specifying emp_id to validate automatic key generation by the trigger.

• Ran a SELECT query to verify that each inserted record received a unique and sequential primary key value.

• Attempted multiple INSERT operations to ensure that no duplicate or NULL primary key values were generated.

• Observed the table contents to confirm that the trigger successfully enforced automatic primary key assignment without manual intervention.

• Analysed the results to ensure data integrity and correctness of the trigger-based automation mechanism.

Input/Output Analysis

SQL Input Queries:

```
-- 1. Create Sequence (acts like auto-increment engine)
CREATE SEQUENCE emp_seq
START 1;

-- 2. Create Table (NO auto increment here)
CREATE TABLE employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    salary NUMERIC(10,2)
);

-- 3. Create Trigger Function
CREATE OR REPLACE FUNCTION assign_emp_id()
RETURNS TRIGGER AS $$
BEGIN
    -- If user doesn't provide ID, generate one
    IF NEW.emp_id IS NULL THEN
        NEW.emp_id := nextval('emp_seq');
    END IF;

    RETURN NEW;
END;
```

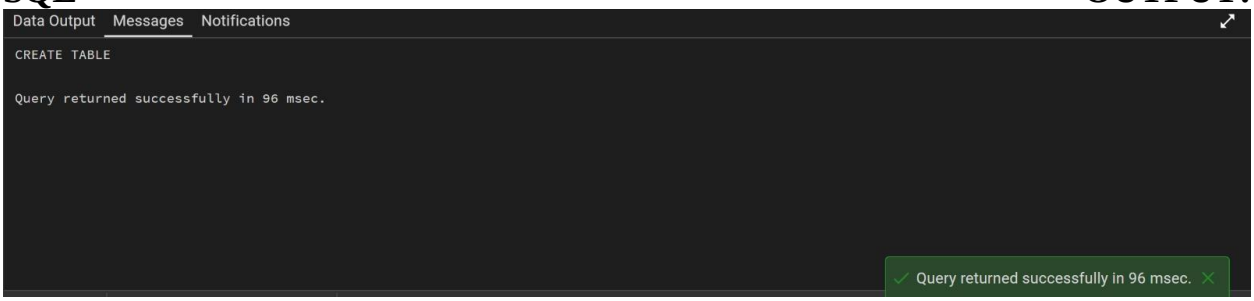
```
$$ LANGUAGE plpgsql;


-- 4. Create Trigger
CREATE TRIGGER before_insert_emp
BEFORE INSERT ON employee
FOR EACH ROW
EXECUTE FUNCTION assign_emp_id();


-- 5. Insert Data (WITHOUT specifying emp_id)
INSERT INTO employee (emp_name, dept_id, salary) VALUES
('A', 101, 1000),
('B', 102, 2000),
('C', 101, 2000),
('D', 102, 3000);


-- 6. Check Data
SELECT * FROM employee;
```

SQL **OUTPUT:**



```
CREATE FUNCTION

Query returned successfully in 150 msec.
```

Query returned successfully in 150 msec.

Total rows: | Query complete 00:00:00.150 | CRLF | Ln 14, Col 1

```
CREATE SEQUENCE

Query returned successfully in 87 msec.
```

Query returned successfully in 87 msec.

Total rows: | Query complete 00:00:00.087 | CRLF | Ln 2, Col 1

Data Output | Messages | Notifications

```
CREATE TRIGGER

Query returned successfully in 82 msec.
```

Query returned successfully in 82 msec.

Data Output | Messages | Notifications

```
INSERT 0 4

Query returned successfully in 80 msec.
```

Total rows: | Query complete 00:00:00.080 | CRLF | Ln 33, Col 1

Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1 of 1

	emp_id [PK] integer	emp_name character varying (50)	dept_id integer	salary numeric (10,2)
1	1	A	101	1000.00
2	2	B	102	2000.00
3	3	C	101	2000.00
4	4	D	102	3000.00

Successfully run. Total query runtime: 406 msec. 4 rows affected.

Total rows: 4 | Query complete 00:00:00.406 | CRLF | Ln 40, Col 1

Learning Outcomes

- After completing this experiment, the learner will be able to:
- Understand the purpose and working of database triggers.
- Implement automated primary key functionality using triggers.
- Ensure data integrity without manual key assignment.
- Apply trigger-based automation in real-world enterprise applications such as Amazon, Flipkart, and Oracle.